

Ex. No. 9**BINARY SEARCH TREES AND TREE TRAVERSALS**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* left;
```

```
    struct Node* right;
```

```
};
```

```
struct Node* createNode(int data) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = data;
```

```
    newNode->left = NULL;
```

```
    newNode->right = NULL;
```

```
    return newNode;
```

```
}
```

```
struct Node* insert(struct Node* root, int data) {
```

```
    if (root == NULL) {
```

```
        root = createNode(data);
```

```
        return root;
    }
    if (data < root->data) {
        root->left = insert(root->left, data);
    } else if (data > root->data) {
        root->right = insert(root->right, data);
    }
    return root;
}
```

```
struct Node* findMin(struct Node* root) {
    while (root->left != NULL) {
        root = root->left;
    }
    return root;
}
```

```
struct Node* delete(struct Node* root, int data) {
    if (root == NULL) {
        return root;
    }
```

```
} else if (data < root->data) {  
    root->left = delete(root->left, data);  
}  
else if (data > root->data) {  
    root->right = delete(root->right, data);  
}  
else {  
    // Case 1: No child  
    if (root->left == NULL && root->right == NULL) {  
        free(root);  
        root = NULL;  
    }  
    // Case 2: One child  
    else if (root->left == NULL) {  
        struct Node* temp = root;  
        root = root->right;  
        free(temp);  
    } else if (root->right == NULL) {  
        struct Node* temp = root;  
        root = root->left;  
        free(temp);  
    }  
}
```

```
// Case 3: Two children
else {
    struct Node* temp = findMin(root->right);
    root->data = temp->data;
    root->right = delete(root->right, temp->data);
}
}
return root;
}
```

```
struct Node* search(struct Node* root, int data) {
    if (root == NULL || root->data == data) {
        return root;
    }
    if (data < root->data) {
        return search(root->left, data);
    }
    return search(root->right, data);
}
```

```
void inorderTraversal(struct Node* root) {  
    if (root != NULL) {  
        inorderTraversal(root->left);  
        printf("%d ", root->data);  
        inorderTraversal(root->right);  
    }  
}
```

```
void preorderTraversal(struct Node* root) {  
    if (root != NULL) {  
        printf("%d ", root->data);  
        preorderTraversal(root->left);  
        preorderTraversal(root->right);  
    }  
}
```

```
void postorderTraversal(struct Node* root) {  
    if (root != NULL) {  
        postorderTraversal(root->left);  
        postorderTraversal(root->right);
```

```
        printf("%d ", root->data);  
    }  
}
```

```
int main() {  
    struct Node* root = NULL;  
  
    int choice, data;  
  
    while (1) {  
        printf("1. Insert\n");  
        printf("2. Delete\n");  
        printf("3. Search\n");  
        printf("4. In-order Traversal\n");  
        printf("5. Pre-order Traversal\n");  
        printf("6. Post-order Traversal\n");  
        printf("7. Exit\n");  
  
        printf("Enter your choice: ");  
  
        scanf("%d", &choice);  
  
        switch (choice) {  
            case 1:  
                printf("Enter the data to insert: ");
```

```
scanf("%d", &data);  
  
root = insert(root, data);  
  
break;
```

case 2:

```
printf("Enter the data to delete: ");  
  
scanf("%d", &data);  
  
root = delete(root, data);  
  
printf("Node deleted\n");  
  
break;
```

case 3:

```
printf("Enter the data to search: ");  
  
scanf("%d", &data);  
  
if (search(root, data) != NULL) {  
    printf("Data found!\n");  
} else {  
    printf("Data not found!\n");  
}  
  
break;
```

case 4:

```
printf("In-order Traversal: ");
```

```
        inorderTraversal(root);

        printf("\n");

        break;

    case 5:

        printf("Pre-order Traversal: ");

        preorderTraversal(root);

        printf("\n");

        break;

    case 6:

        printf("Post-order Traversal: ");

        postorderTraversal(root);

        printf("\n");

        break;

    case 7:

        exit(0);

    default:

        printf("Invalid choice!\n");

    }

}

return 0;
```

}